

ENTITY RELATIONSHIP DIAGRAM

Project Title: **Smart Parking System**

Group Number: **04**

Group Members:

- Sherbano Naveed 24p-0599
- Ayesha Anees 24p-0595
- Ayesha Zulfiqar 24p-0663

Explanation of ERD:

Entities and Their Roles:

User is the central identity table of the entire system. Every person who interacts with the system, whether they are parking a car or managing a lot must first exist as a User record. It stores the essential personal information needed for authentication and contact.

- User_ID is the primary key that uniquely identifies every person in the system.
- It stores Name, Email, Password, and Phone_Number as basic profile attributes.
- No one can be a Driver or an Admin without first being a User

Driver and Admin are subtypes of User, implementing what is known in DBMS as an ISA (is-a) or specialization relationship. Rather than duplicating personal information, both tables simply reference the User table using the same User_ID as both their primary key and foreign key.

- A Driver is a user who parks vehicles. The Driver table on its own holds no extra attributes, all personal data lives in User.
- An Admin is a user who manages parking lots and oversees the system.
- This design avoids data redundancy and ensures that deleting a User automatically removes their Driver or Admin record through cascading.

- One important constraint: a single user cannot be both a Driver and an Admin at the same time under this design, since the subtype relationship is one-to-one and exclusive.

Vehicle represents the cars, bikes, or any other vehicles registered in the system by drivers. Since a single driver may own multiple vehicles (a family car and a motorbike, for example), this is modeled as a one-to-many relationship from Driver to Vehicle.

- Vehicle_ID is the primary key.
- Vehicle_Number stores the license plate and is marked unique no two vehicles can share the same plate.
- Vehicle_Type is critical because it is later used to look up the correct hourly parking rate.
- User_ID is the foreign key linking each vehicle back to its owning driver.
- A vehicle can exist in the system without any parking history yet it does not need a ticket to be valid.

Parking_Lot represents a physical parking facility managed by an admin. Each lot has a location and a total slot count, and is associated with the admin responsible for it.

- Lot_ID is the primary key.
- Total_Slots records the capacity of the lot, which is useful for availability checks at the application level.
- User_ID is a foreign key referencing the Admin table — it identifies which admin oversees this lot.
- An admin may manage zero lots initially (for example, a newly created admin account), which is why this relationship is optional on the Admin side.

Parking_Slot represents an individual parking space within a lot. Every slot belongs to exactly one lot, and a lot must contain at least one slot for the relationship to be meaningful.

- Slot_ID is the primary key.

- Slot_Number identifies the slot within its lot (e.g., A1, B3). A composite unique constraint on (Lot_ID, Slot_Number) ensures no two slots in the same lot share the same number.
- Slot_Status tracks whether the slot is currently Available, Occupied, or Reserved.
- Lot_ID is the foreign key linking the slot to its parent lot.
- A slot can exist without any parking history, an empty slot is perfectly valid and should not require a ticket record.

Parking_Rate is a reference table that defines how much to charge per hour based on vehicle type. It exists independently of any specific parking event and is referenced whenever a bill needs to be calculated.

- Rate_ID is the primary key.
- Vehicle_Type is unique in this table, each vehicle type has exactly one rate.
- Rate_Per_Hour stores the monetary amount charged per hour for that vehicle type.
- This table makes the system flexible: changing a rate requires updating just one record here rather than modifying historical bills.

Parking_Ticket is the core transactional entity of the system. It is created every time a vehicle enters a parking slot and serves as the event record tying a vehicle to a slot for a specific time period.

- Ticket_ID is the primary key.
- Entry_Time is mandatory and recorded when the vehicle arrives. Exit_Time is initially null and is filled in when the vehicle leaves.
- Vehicle_ID is a foreign key to Vehicle, each ticket must be associated with a known vehicle.
- Slot_ID is a foreign key to Parking_Slot, each ticket records exactly which slot was used.

- Both foreign keys are mandatory on the ticket side, meaning a ticket cannot exist without knowing which vehicle used which slot.
- A database-level check constraint enforces that Exit_Time must always be later than Entry_Time when provided.

Billing is generated after a vehicle exits and a ticket is closed. It calculates the total charge for the parking session. The relationship between Billing and Parking_Ticket is strictly one-to-one: one ticket produces exactly one bill, no more, no less.

- Bill_ID is the primary key.
- Ticket_ID is a foreign key to Parking_Ticket and is marked unique, which is what enforces the one-to-one constraint at the database level.
- Rate_ID is a foreign key to Parking_Rate, it records which rate was applied when the bill was calculated, preserving a historical reference even if the rate changes later.
- Amount stores the final computed charge, and Bill_Time records when the bill was generated.

Payment records how a bill was actually settled by the driver. Like Billing to Ticket, the relationship between Payment and Billing is one-to-one — one bill is settled by exactly one payment record.

- Payment_ID is the primary key.
- Bill_ID is a foreign key to Billing and is also marked unique to enforce the one-to-one constraint.
- Payment_Method records how the driver paid — Cash, Card, Online, or Wallet.
- Payment_Time records when the transaction occurred.

- `Payment_Status` tracks the outcome: Pending, Completed, Failed, or Refunded, making it possible to handle failed or disputed transactions cleanly.

